SOMAIYA
VIDYAVIHAR
K J Somaiya Institute of Technology

An Autonomous Institute Permanently Affiliated to the University of Mumbai

DEPARTMENT OF INFORMATION TECHNOLOGY

Course Name and Code: Data Structures Lab (ITL302)
Semester: III (SYIT)
Academic Year: 2026-27 (Odd Semester)
Name: Rashi Dawda
Roll no: 23

Experiment No. 08

Aim: Implementation of the Bubble Sort technique to sort an array in ascending or descending order, as chosen by the user.

Objectives:
To introduce the concepts of data structures and analysis procedure.
To understand sorting techniques and analyse their time and space complexity.

Theory: Introduction to Sorting
Sorting is the process of arranging the elements of a collection in a particular order, either ascending (small to big) or descending (big to small). Sorting makes later operations such as searching, merging and reporting faster and simpler.

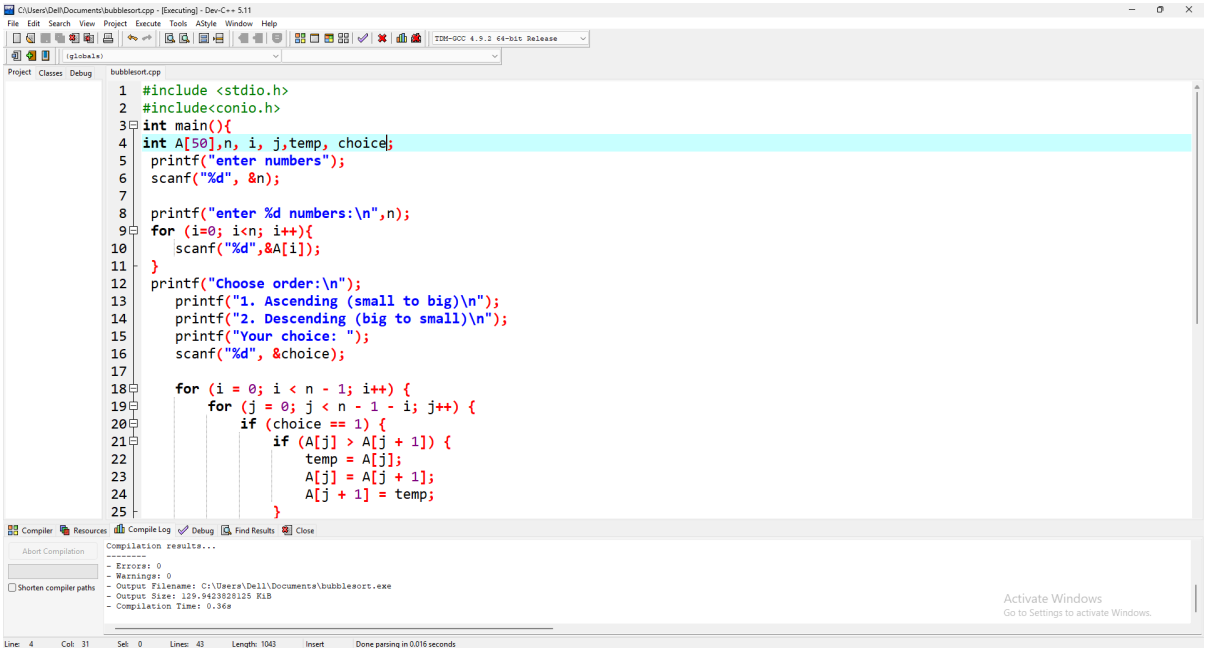
Bubble Sort
Bubble sort repeatedly compares two adjacent elements and swaps them if they are in the wrong order. After the first pass the largest element (for ascending order) has "bubbled" to the last position, after the second pass the second largest is in place, and so on. For n elements, n-1 passes are enough, and each pass can ignore the elements that are already in their final position.

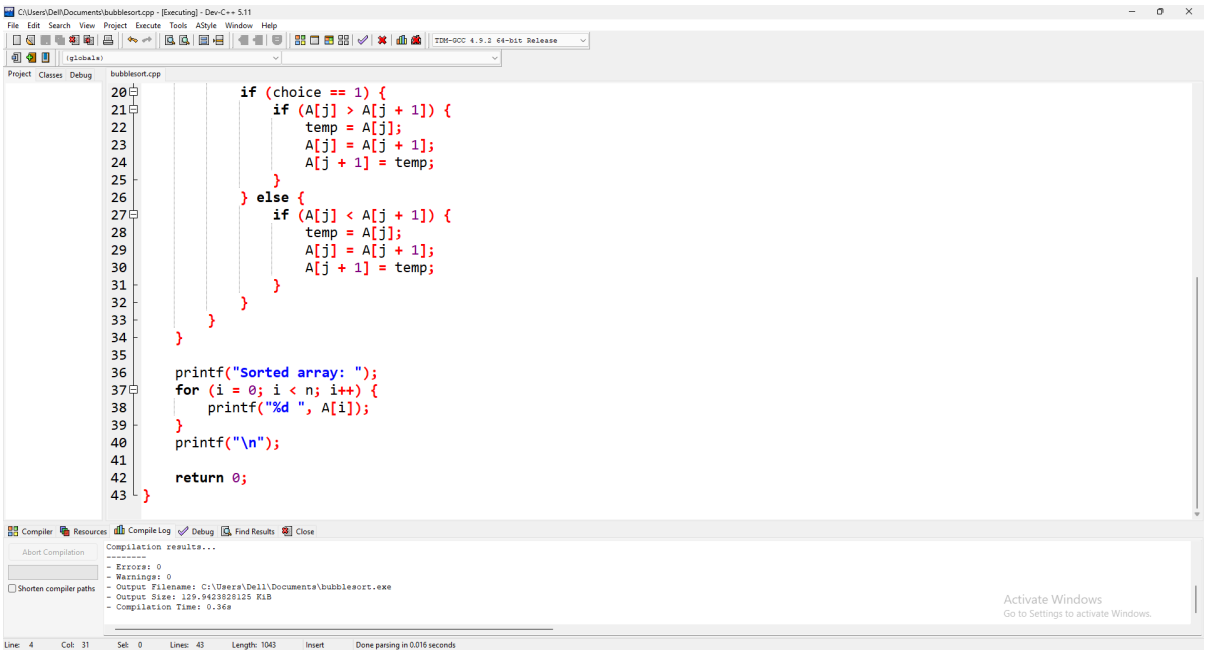
`int A[50];` is the array that stores the input numbers (up to 50).
`int n;` is the number of elements entered by the user.
`int i, j;` are the loop counters: i counts the passes and j walks through adjacent pairs.
`int temp;` is the temporary variable used to swap two elements.
`int choice;` stores the order selected: 1 for ascending, 2 for descending.

Algorithm
1. Declare A[50], n, i, j, temp and choice.
2. Read n and then read n elements into A[0] ... A[n-1].
3. Read choice (1 = ascending, 2 = descending).
4. For i = 0 to n-2 (one pass per iteration), repeat steps 5 and 6.
5. For j = 0 to n-2-i, compare A[j] with A[j+1].
6. If choice == 1 and A[j] > A[j+1], swap them using temp. Otherwise, if A[j] < A[j+1], swap them using temp.
7. After all passes, print the array from A[0] to A[n-1].

Complexity
Time: $O(n^2)$ in the average and worst case, since the two nested loops make $n(n-1)/2$ comparisons. As written (without an early-exit flag) the loops run fully even on sorted input, so the best case is also $O(n^2)$.
Space: O(1) extra space, since the array is sorted in place and only temp is needed. Bubble sort is also stable, because equal elements are never swapped.

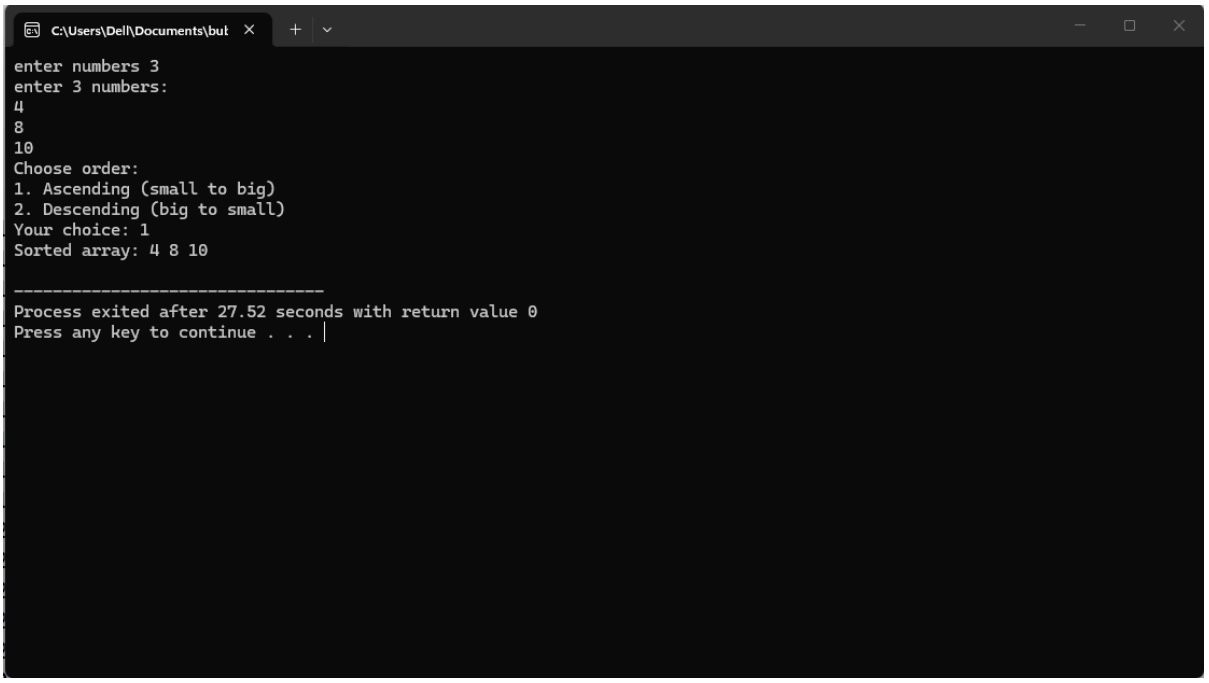
Code:


C:\Users\Dell\Documents\bubblesort.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Project Classes Debug
bubblesort.cpp
#include <stdio.h>
#include<conio.h>
int main(){
int A[50],n, i, j,temp, choice;
printf("enter numbers");
scanf("%d", &n);
printf("enter %d numbers:\n",n);
for (i=0; i<n; i++){
scanf("%d",&A[i]);
}
printf("Choose order:\n");
printf("1. Ascending (small to big)\n");
printf("2. Descending (big to small)\n");
printf("Your choice: ");
scanf("%d", &choice);
for (i = 0; i < n - 1; i++) {
for (j = 0; j < n - 1 - i; j++) {
if (choice == 1) {
if (A[j] > A[j + 1]) {
temp = A[j];
A[j] = A[j + 1];
A[j + 1] = temp;
}
Compiler Resources Compile Log Debug Find Results Close
Abort Compilation
Shorten compiler paths
Compilation results...
- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\Dell\Documents\bubblesort.exe
- Output Size: 129.9423828125 KiB
- Compilation Time: 0.36s
Activate Windows
Go to Settings to activate Windows.
Line: 4 Col: 31 Sel: 0 Lines: 43 Length: 1043 Insert Done parsing in 0.016 seconds


C:\Users\Dell\Documents\bubblesort.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Project Classes Debug
bubblesort.cpp
20 if (choice == 1) {
21 if (A[j] > A[j + 1]) {
22 temp = A[j];
23 A[j] = A[j + 1];
24 A[j + 1] = temp;
25 }
26 } else {
27 if (A[j] < A[j + 1]) {
28 temp = A[j];
29 A[j] = A[j + 1];
30 A[j + 1] = temp;
31 }
32 }
33 }
34 }
35
36 printf("Sorted array: ");
37 for (i = 0; i < n; i++) {
38 printf("%d ", A[i]);
39 }
40 printf("\n");
41
42 return 0;
43 }
Compiler Resources Compile Log Debug Find Results Close
Abort Compilation
Shorten compiler paths
Compilation results...

- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\Dell\Documents\bubblesort.exe
- Output Size: 129.9423828125 KiB
- Compilation Time: 0.36s
Activate Windows
Go to Settings to activate Windows.
Line: 4 Col: 31 Sel: 0 Lines: 43 Length: 1043 Insert Done parsing in 0.016 seconds

Output:



Conclusion: In this experiment I learnt how bubble sort arranges an array by repeatedly comparing neighbouring elements and swapping them when they are out of order. Each pass pushes one more element into its final position at the end of the array, which is why the inner loop can stop at n-1-i instead of going through the whole array every time. Using a single choice variable let the same two loops sort in both ascending and descending order, with only the comparison (> or <) changing. The program compiled with no errors or warnings and produced the sorted output for the sample input. Bubble sort is easy to write and needs no extra memory, but its $O(n^2)$ time makes it slow for large inputs, and without an early-exit flag it does the same work even when the array is already sorted. For bigger data, algorithms like merge sort or quick sort are the better choice.

Outcome: Apply the concepts of sorting techniques and analyse their time and space complexity.